

Property-Based Testing

17-313 Fall 2026

Foundations of Software Engineering

<https://cmu-17313q.github.io>

Eduardo Feo Flushing

Administrivia

- Sprint 2 season.
- Features should not be purely cosmetic or arbitrary.
- We expect at least one high-impact OpenCode feature that spans multiple levels of the development stack: CLI/desktop UI, agent or tool logic, and provider integrations.
- Some team features currently lack a UI or leave the user experience unclear.
- Some slides relied too much on AI-generated content. Keep the content concise and clear.
- OpenCode as a tool for learning software engineering?

Learning Goals

- Explain what a **test oracle** is, and distinguish the oracle from the mechanism used to check it.
- Explain why crash detection is a useful but **weak oracle** for fuzz testing.
- Explain **property-based testing (PBT)** as generated inputs plus properties that act as automated oracles.
- Identify useful classes of properties: invariants, algebraic, metamorphic, differential, and conditional properties.
- Design a generator and a property for a small program such as the Wordle matching function.
- Recognize that PBT does not eliminate the oracle problem: bad properties or bad generators still produce weak tests.

Outline

- **From Fuzzing to PBT**
- Good properties
- Generate & shrink
- Wordle activity

Recap: Fuzzing Needs an Oracle

Recap from last lecture



Fuzzing automates input generation, exploring far more cases than anyone could pick by hand.

But how did we know whether the result is correct?

NOTES

Automated input generation improves exploration: fuzzing found real crashes in real software. But input generation alone is not enough to detect many bugs. Generating an input and running the program tells us nothing about whether the result was correct unless we also have a way to judge it.

Recap: What Is a Test Oracle?

Test oracle: the automated mechanism we use to decide whether an execution is correct.

"Was this execution correct or incorrect?"

Program satisfies domain-specific properties

↑ stronger

Assertions / invariants report no illegal state

↑ stronger

Program does not crash

Crash detection is useful, but it's the weakest oracle we have.

NOTES

An oracle is not the name of a library or an object in your program. It is whatever lets us decide whether this particular execution was correct. "Did not crash" does not mean "correct." It is the weakest useful oracle, but it is generic: it applies to any input, which is exactly why it pairs so well with fuzzing. Assertions and invariants make more incorrect behavior observable without needing test-specific knowledge. Semantic, domain-specific properties are the strongest, but they take real effort to design.

Activity: From Expected Output to Properties

`xs` → `sort(xs)`

Instead of specifying the exact expected output, what must be true about the relationship between input and output, for any input array?

Example: `xs.length === sort(xs).length` is one such property. Find at least one more.

2–3 minutes, in pairs.

NOTES

Rather than specifying an exact expected output, look for relationships that must hold between any input and its output. This is the core shift from example-based thinking to property-based thinking.

Debrief: From Expected Output to Properties

- The output is sorted. (property of the output)
- The output has the same length as the input. (input/output relationship)
- The output contains the same elements, with the same multiplicities, as the input. (input/output relationship)

Are these properties individually enough to tell us that sort is correct?

```
function sort(xs) {  
  return [];  
}
```

Satisfies "the output is sorted." Violates "the output contains the same elements as the input."

A single property may only capture part of the specification. We often need multiple complementary properties.

We often don't know the exact expected output for every generated input, but we know relationships that must hold between input and output.

NOTES

No single property is automatically sufficient; weak properties can be satisfied by broken implementations. Good property-based testing usually combines several properties, and this same idea returns later as invariants, algebraic, metamorphic, differential, and conditional properties.

What Is a Property?

Property: a general statement about behavior that should hold across many inputs or executions.

```
assert(  
    matchGuess(secret, guess).length === 5  
);
```

```
assert(balance >= 0);
```

For every valid `secret / guess` ,
`matchGuess` returns 5 colors.

The balance should never be negative.

Preconditions, postconditions, and invariants are all properties.

Today we use properties as the basis for automatically generated tests.

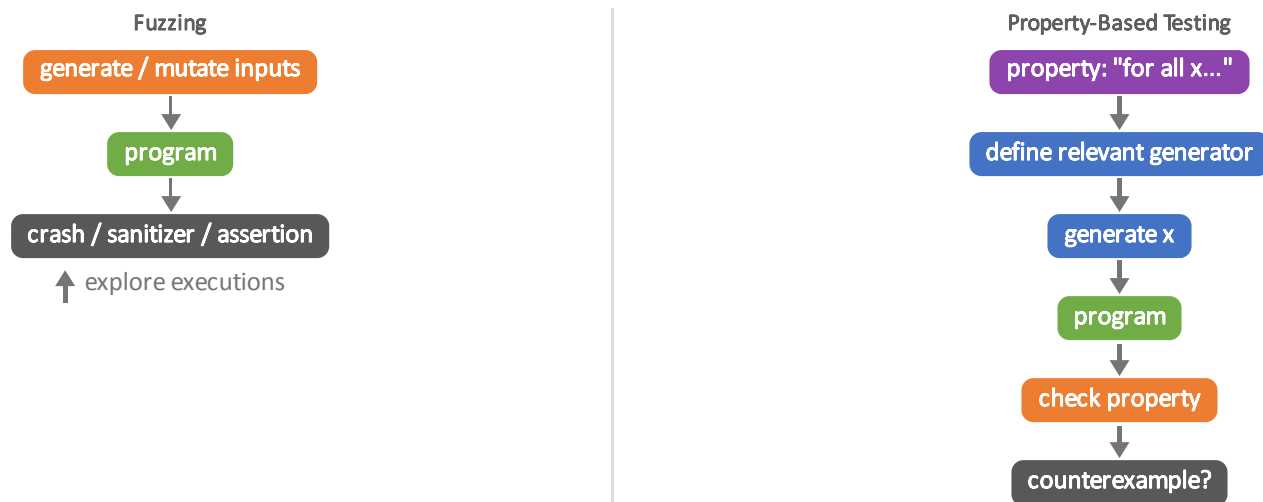
NOTES

A precondition, postcondition, or invariant from the previous lecture is already a property: a general statement about behavior that should hold. In the previous lecture, we used them as assertions inside the program to detect invalid states. In PBT, we use similar properties as test oracles in the test code, checking them across many generated executions.

The Bridge to Property-Based Testing

Last time, we already had both ingredients: generated inputs and assertions.

PBT changes what drives the test: the property becomes the specification we are trying to falsify.



PBT starts from a property and searches for a counterexample.

NOTES

Fuzzing explores executions and checks a generic, weak oracle after the fact. Property-based testing flips the direction: we start from a property, the specification we want to falsify, and that property shapes the generator we design.

What Is Property-Based Testing?

- Popularized by the Haskell library QuickCheck.
- Properties are **testable specifications written in code**.

A PBT framework repeatedly tries to find a counterexample:

1. Generate an input.
2. Run the program.
3. Check the property.
4. Stop when the property is falsified.

In this course: fast-check for TypeScript/JavaScript

NOTES

A property is an automated oracle. Properties normally live in test code, while assertions may also exist in production code. PBT checks a property across many generated inputs rather than one hand-picked example.

Example-Based vs. Fuzzing vs. Property-Based Testing

Technique	Typical emphasis
Example-based testing	Specific scenarios and expected results
Fuzzing	Exploring a large/unexpected input space
Property-based testing	Trying to falsify general properties

The boundaries are not strict, but each technique has a different main goal

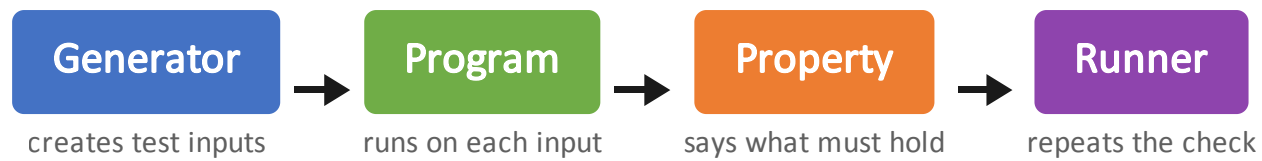
NOTES

PBT and fuzzing overlap; fuzzing can use semantic properties, and PBT can also detect crashes. The distinction is not "random vs. not random," it is what each technique treats as central. Example-based tests and PBT are complementary, not competitors.

Outline

- From Fuzzing to PBT
- **Good properties**
- Generate & shrink
- Wordle activity

Anatomy of a Property-Based Test



Later: shrinking counterexamples.

NOTES

The generator defines what part of the input space we explore. The property decides what counts as failure. The runner repeats this across many generated inputs; later we will see how it also shrinks a failing input to a small, useful counterexample.

A Property-Based Test in `fast-check`

```
fc.assert(  
  fc.property(fc.array(fc.integer()), xs => {  
    const ys = sort(xs);  
    return isNonDecreasing(ys);  
  })  
);
```

Generator : `fc.array(fc.integer())` explores arrays of integers.

Program under test : `sort(xs)` is the function we are testing.

Property : `isNonDecreasing(ys)` is the oracle.

Runner : `fc.assert` repeats this across many generated `xs`.

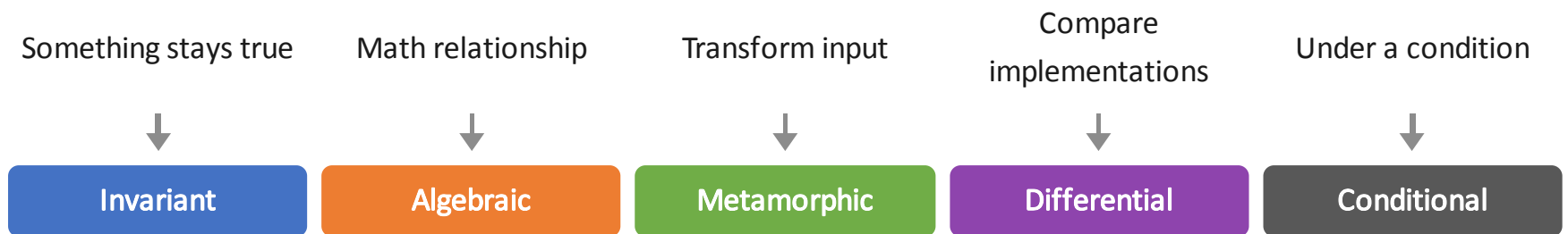
NOTES

The framework generates values for `xs`. The callback states the property. A returned false means the framework found a counterexample.

Property Patterns: A Toolbox

How do we find good properties?

We don't have to invent every property from scratch. There are recurring patterns we can look for.



Each pattern answers the same question: where can a property come from?

NOTES

These five recurring patterns cover most cases: invariants, algebraic relationships, metamorphic relationships, differential comparison, and conditional properties. These are five ways to come up with useful properties.

Invariants: What Must Always Stay True?

Invariant → Algebraic → Metamorphic → Differential → Conditional

- Sorting preserves the multiset of elements.
- A cart total is never negative.
- Wordle always returns exactly five color results for valid five-letter inputs.
- Every Wordle result is one of **G**, **Y**, **B**.

NOTES

An invariant captures something that must stay unchanged or remain within valid bounds. Invariants are often the easiest PBT properties to identify. Assertions and PBT properties frequently express the same kind of invariant at different places in the codebase.

Algebraic Properties: Round Trip, Idempotence, Commutativity

Invariant → Algebraic → Metamorphic → Differential → Conditional

These come from a mathematical relationship between operations, not from computing an exact expected output.

Round trip

```
decode(encode(x)) == x
```

Idempotence

```
normalize(normalize(x))  
== normalize(x)
```

Commutativity

```
combine(a, b)  
== combine(b, a)
```

These relationships are useful properties only when they are guaranteed by the specification.

NOTES

These properties avoid hard-coding expected outputs: the operation itself gives us a relationship that should hold across many inputs. Not every function should satisfy every mathematical property, so check against the specification before assuming one applies.

Metamorphic Properties

Invariant → Algebraic → **Metamorphic** → Differential → Conditional

Change the input in a way that should preserve a relationship.

- Sorting: shuffling the input should not change the sorted output.
- Cart: reordering items should not change the total.
- Wordle matching: consistently renaming letters in both the secret and the guess should preserve the color pattern.



NOTES

When we cannot easily know the correct answer, we may still know how the answer should change, or stay unchanged, when we transform the input. Metamorphic testing compares related executions rather than comparing one output with a known answer. For Wordle, this property applies to the matching function, not dictionary validation.

Differential Properties

Invariant → Algebraic → Metamorphic → **Differential** → Conditional

Run two versions → compare results.

- New optimized sorter vs. slow trusted sorter.
- Old implementation vs. refactored implementation.
- Wordle production matcher vs. a small, obviously-correct reference matcher.



NOTES

A trusted implementation can act as the oracle for another implementation. Differential testing is useful whenever a reference implementation exists.

Conditional Properties

Invariant → Algebraic → Metamorphic → Differential → **Conditional**

If A happens, B must hold.

- **Wordle:** If `secret === guess`, then the output must be `GGGGG`.
- If a withdrawal amount exceeds the balance, the operation must fail without changing the balance.

$$A \Rightarrow B$$

That completes the toolbox: invariants, algebraic, metamorphic, differential, and conditional properties.

NOTES

Some properties only make sense when a precondition holds. Preconditions are part of the property, and we need to think about how often generated data actually satisfies those preconditions.

Outline

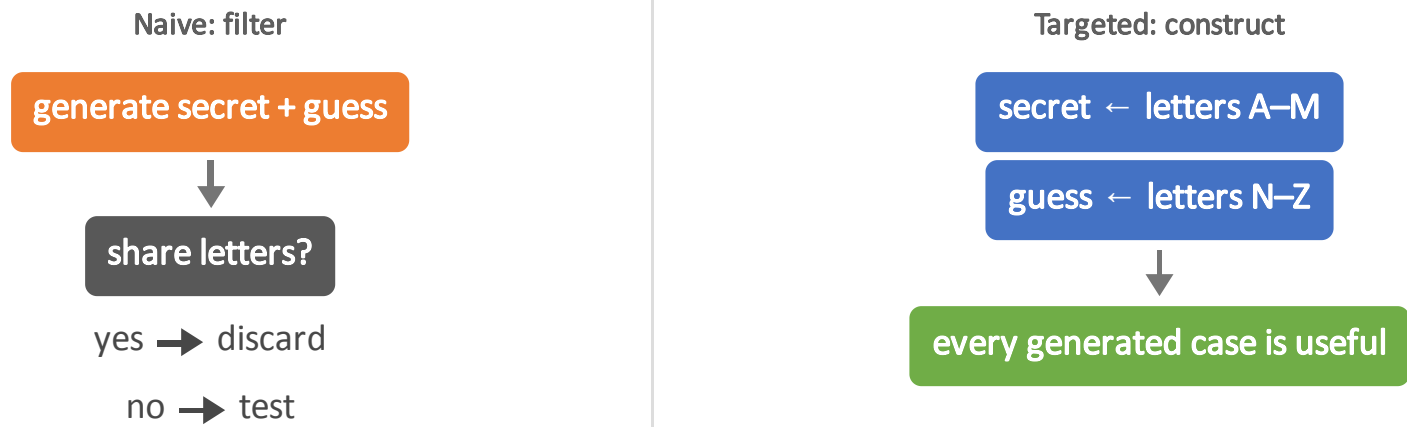
- From Fuzzing to PBT
- Good properties
- **Generate & shrink**
- Wordle activity

Filtering vs. Targeted Generators

We've talked a lot about properties. But remember: PBT still depends on the inputs we generate.

A great property is useless if our generator never produces the interesting cases.

Example Property: "If `secret` and `guess` share no letters, every result must be `B`."



NOTES

When a property applies only to some inputs, we can generate broadly and discard irrelevant cases, or generate the relevant cases directly. Generator design matters as much as property design: a test that rarely reaches the interesting case is weak even if the property is correct. Filtering by discarding non-interesting inputs is easy but can waste many generated cases; a targeted generator does more setup but every case (input) will be useful.

Targeted Generators in Practice

```
const wordFrom = (letters: string) =>
  fc.array(fc.constantFrom(...letters), { minLength: 5, maxLength: 5 })
    .map(cs => cs.join(''));
const secret = wordFrom('ABCDEFGHIJKLM');
const guess = wordFrom('NOPQRSTUVWXYZ');
fc.assert(
  fc.property(secret, guess, (secret, guess) =>
    matchGuess(secret, guess).every(c => c === 'B')
  )
);
```

For example, it might generate:



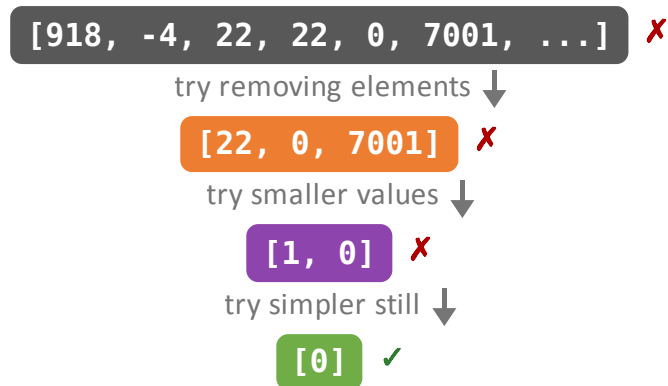
Don't generate arbitrary inputs and hope they satisfy the condition. Generate inputs that satisfy the condition by construction.

NOTES

Because secret is drawn only from A-M and guess only from N-Z, the two words can never share a letter: every generated pair is a useful test case, with no wasted or discarded attempts. The generator is now doing real work, constructing inputs with a desired structural relationship, rather than merely generating arbitrary values and hoping.

Shrinking: From "It Failed" to "Here Is Why"

The framework repeatedly tries simpler versions of the failing input. If the property still fails, it keeps the simpler input and continues shrinking.



Reported counterexample: `[1, 0]` , because `[0]` no longer fails

A simpler counterexample that still fails the property, not necessarily the smallest possible one.

In fast-check, generators know how to shrink the values they generate.

NOTES

PBT frameworks do more than find a failing input: they try to simplify it. Randomly generated failures can be hard to understand, and shrinking turns them into small, reproducible counterexamples. This is a major practical difference between a PBT framework and a simple random loop.

Outline

- From Fuzzing to PBT
- Good properties
- Generate & shrink
- **Wordle activity**

Activity: Property-Based Testing for Wordle

Which of These Are Valid Properties?

```
matchGuess(secret: string, guess: string): Color[]
```

For each: is it a valid property of `matchGuess` ? If valid, what type of property is it?

- A. Output always has length 5.
- B. Every output is one of `G`, `Y`, `B`.
- C. If `secret === guess`, every output is `G`.
- D. `matchGuess(secret, guess) === matchGuess(guess, secret)`
- E. For each letter `L`, the number of `G` / `Y` matches for `L` cannot exceed its occurrences in the secret.
- F. Renaming every letter consistently in both inputs should preserve the color pattern.

NOTES

A property must come from the specification, not from intuition alone. You should reason about validity first and classify second.

Debrief: Which Properties Are Valid?

Statement	Valid?	Type
A. Output always has length 5	✓	Invariant
B. Every output is G / Y / B	✓	Invariant
C. If secret === guess , every output is G	✓	Conditional property

Statement	Valid?	Type
D. Swapping secret / guess gives the same result	✗	Not symmetric
E. G / Y matches for a letter ≤ its occurrences in secret	✓	Invariant / domain
F. Consistently renaming letters preserves the color pattern	✓	Metamorphic

D looks like commutativity (algebraic), but Wordle matching does not satisfy it.

Knowing a property pattern doesn't mean your function should satisfy it. The property has to come from the specification.

A and B are valid, but are they strong enough alone? matchGuess could return BBBBB for every input and still satisfy both.

Valid property $\neq$ sufficient oracle.

NOTES

D is a useful trap: Wordle matching is not symmetric, so a plausible-sounding property can still be wrong if it does not come from the specification. A and B alone are also too weak: a function that always returns BBBBB satisfies both yet is completely broken, which is exactly the oracle-strength problem from earlier in the lecture.

Takeaways

1. An oracle tells us whether an execution is correct.
2. "The program did not crash" is useful, but weak.
3. Properties give us stronger, more general oracles.
4. Good PBT needs good properties and good generators.
5. Shrinking turns failures into useful counterexamples.



PBT: generate many inputs and look for one that breaks a property.